

VFP Nested-Virtualization Lab

Setup & Run Guide — L1 Windows Server host + nested L2 Ubuntu VMs

Summary: These scripts bring up an **SDN-style nested-virtualization** environment on physical devboxes with Hyper-V support, to **reproduce MSRC bugs**. They stand up an **L1 Windows Server Hyper-V VM** on the devbox and nested **L2 Ubuntu VMs** on that L1 host, with the **VFP switch extension** enabled.

MobaXterm is installed and a session is added for each Linux VM, so you can paste commands and drag-and-drop PoC files (Python, .c, etc.) from Windows into Linux. The scripts also program the **VFP rules** that enable **Linux↔Linux**, **Linux↔Windows-gateway**, and **Linux↔Internet** connectivity.

Overview

This guide walks through building a nested Hyper-V lab. You copy a Windows Server **.vhdx**, run **Create-L1Vm-Windows.ps1** to build a fully self-configuring L1 Windows SDN/VFP host, then RDP into it and run **Create-L2Vm-Linux.ps1** to spin up nested L2 Ubuntu VMs from a cloud image in about 30 seconds each.

Credentials

L1 VM (Windows)	Administrator	Admin@123
L2 VM (Ubuntu)	ubuntu	ubuntu@123

Step-by-step

Step 1 — Copy the base VHDX to a local directory

Copy a **serverdatacenter .vhdx** (any latest build) from winbuilds to a local folder such as **C:\VHDS**:

```
Source: \\winbuilds\release\rs_iof\29655.1000.260819-1437\amd64fre\vhdx\  
        vhdx_server_serverdatacenter_en-us\ ... .vhdx  
Dest:   C:\VHDS\
```

Tip: instead of copying manually, you can pass **-Branch rs_iof** and the script robocopies the latest build for you.

Step 2 — Create the L1 Windows VM (~8–10 mins)

Run in an **elevated** PowerShell session on the Hyper-V host:

```
PS> .\Create-L1Vm-Windows.ps1 -L1VMName "L1-RsIoF-2025" `  
    -VHDXPath "C:\VHDS\29655.1000.amd64fre.rs_iof.260819-1437_  
    server_serverdatacenter_en-us.vhdx"
```

Step 3 — RDP into L1 and create an L2 Ubuntu VM (~30 seconds)

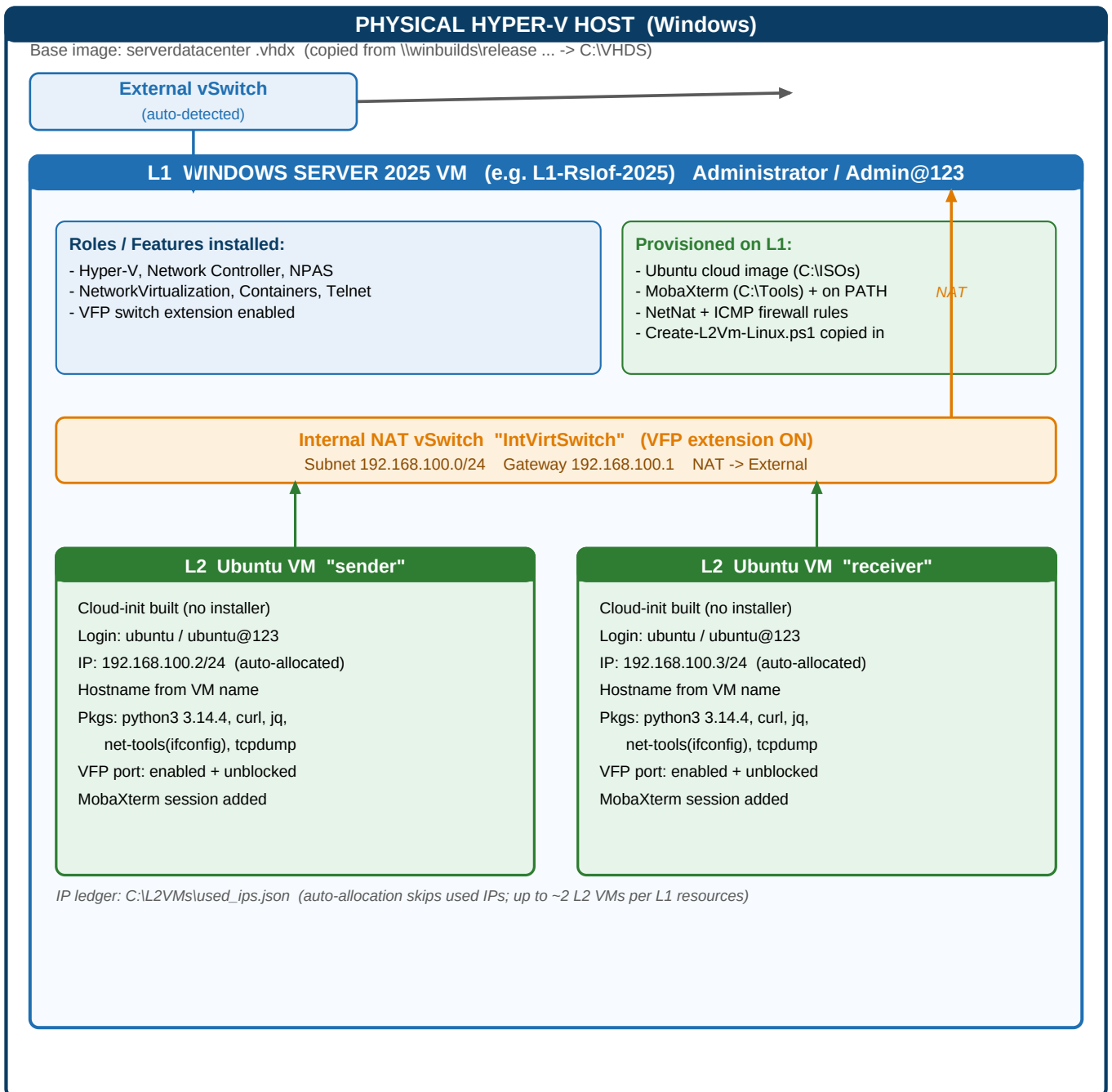
Connect (RDP) to the L1 VM from Hyper-V, open **PowerShell as Administrator**, and run:

```
C:\Windows\system32> .\Create-L2Vm-Linux.ps1 -L2VMName "sender"
```

- A **MobaXterm** session for the new VM pops up automatically — use it to paste commands and drag & drop files onto the Linux VM.
- The Linux VM ships with **python3 (3.14.4)**, **curl**, **ifconfig** (net-tools), **jq**, and **tcpdump** preinstalled.
- The IP is auto-allocated from 192.168.100.0/24 and recorded, so a second run (e.g. **-L2VMName "receiver"**) gets the next free IP.
- You can create about **2 Linux VMs** within the resources allocated to the L1 Windows VM.

Resulting infrastructure

After all scripts run, the lab looks like this:



What Create-L1Vm-Windows.ps1 does

- Creates a new **Windows Server 2025** VM from the .vhdx with hardcoded configuration and password (unattended first boot).
- Installs required roles/features: **Hyper-V, Network Controller, Telnet, Containers, NPAS, NetworkVirtualization**.
- Creates the **internal switch**, enables the **VFP extension**, enables **NAT**, and adds **ICMP firewall** rules. (Reboots so the VFP driver **vfpxext** starts.)
- Downloads the **cloud-init Ubuntu image** and **MobaXterm**, and puts MobaXterm on PATH.
- Copies **Create-L2Vm-Linux.ps1** into the L1 VM.

What Create-L2Vm-Linux.ps1 does

- Creates a **Linux VM** from the earlier-downloaded cloud-init image, with username and password hardcoded (no installer, boots straight into Ubuntu).
- Installs required packages: **curl, jq, net-tools (ifconfig), tcpdump, python3**.
- Installs **VFP rules** on the new VM's VFP port (enable-port → unblock → ACLs).
- Adds the new VM to a **MobaXterm** session.

Scripts: Create-L1Vm-Windows.ps1, Create-L2Vm-Linux.ps1 • Docs: Create-L1Vm-Windows.md, Create-L2Vm-Linux.md